

Date et heure JavaScript

Dans ce chapitre, vous apprendrez à utiliser la date et l'heure en JavaScript.

Utilisation de l'objet Date

L'objet Date est un objet JavaScript intégré. Il vous permet d'obtenir l'heure locale de l'utilisateur en accédant à l'horloge du système via le navigateur. L'objet Date fournit également plusieurs méthodes pour gérer, manipuler et formater les dates et les heures.

Création d'un objet de date

Avant de commencer à travailler avec la date et l'heure, nous devons créer un objet Date. Contrairement à d'autres objets intégrés, tels que des tableaux ou des fonctions, les dates n'ont pas de forme littérale correspondante : tous les objets de date doivent être créés à l'aide de la fonction constructeur Date qui est Date().

Il existe quatre façons différentes de créer un objet Date en JavaScript :

- La syntaxe `new Date()`
- La syntaxe `new Date(year, month...)`
- La syntaxe `new Date(dateString)`
- La syntaxe `new Date(milliseconds)`

La syntaxe `new Date()`

Vous pouvez simplement déclarer un nouvel objet `Date` sans initialiser sa valeur. Dans ce cas, la valeur de date et d'heure sera définie sur la date et l'heure actuelles sur l'appareil de l'utilisateur sur lequel le script est exécuté.

```
let d = new Date();  
document.write(d);
```

La syntaxe `new Date(year, month...)`

Vous pouvez également initialiser un objet Date en transmettant les paramètres suivants séparés par des virgules : année, mois, jour, heures, minutes, secondes et millisecondes.

Les paramètres **année** et **mois** sont obligatoires, les autres paramètres sont facultatifs.

```
let d = new Date(2018, 0, 31, 14, 35, 20, 50);  
document.write(d);
```

Cette date représente en réalité le 31 janvier 2018 à 14:35:20 et 50 millisecondes. Vous pouvez ignorer la partie heure et spécifier uniquement les parties années et mois si vous le souhaitez.

Utilisation de numéros 6, 4, 3 ou 2

6 chiffres précisent l'année, le mois, le jour, l'heure, la minute, la seconde

```
const d = new Date(2018, 11, 24, 10, 33, 30);
```

5 chiffres indiquent l'année, le mois, le jour, l'heure et la minute :

```
const d = new Date(2018, 11, 24, 10, 33);
```

4 chiffres indiquent l'année, le mois, le jour et l'heure :

```
const d = new Date(2018, 11, 24, 10);
```

3 chiffres indiquent l'année, le mois et le jour :

```
const d = new Date(2018, 11, 24);
```

2 chiffres précisent l'année et le mois :

```
const d = new Date(2018, 11);
```

La syntaxe `new Date(dateString)`

JavaScript vous permet également de créer un objet `Date` en transmettant la chaîne représentant une date, ou une date et une heure, comme illustré dans l'exemple suivant :

```
const d = new Date("31 January 2018");  
document.write(d);
```

Cette date représente le 31 janvier 2018. Vous pouvez également spécifier des chaînes telles que 31 janvier 2018, ou l'une des nombreuses variantes valides, JavaScript s'en chargera automatiquement.

La syntaxe `new Date(milliseconds)`

Vous pouvez également définir un objet Date en passant le nombre de millisecondes depuis le 1er janvier 1970 à 00:00:00 GMT. Cette époque est connue sous le nom d' époque UNIX car 1970 est l'année où le système d'exploitation UNIX a été officiellement introduit. Voici un exemple :

```
const d = new Date(1517356800000);  
document.write(d);
```

La date ci-dessus représente le mercredi 31 janvier 2018 05:30:00 GMT+0530.

Une fois que vous avez créé une instance de l'objet Date, vous pouvez utiliser ses méthodes pour effectuer diverses tâches, telles que l'obtention de différents composants d'une date, la définition ou la modification d'une valeur de date et d'heure individuelle, etc. Ces méthodes sont décrites en détail dans les slides suivants.

Création des chaînes de date et d'heure

L'objet JavaScript Date fournit plusieurs méthodes, telles que `toDateString()`, `toLocaleDateString()`, etc. pour générer des chaînes de date dans différents formats. Voici un exemple :

```
const d = new Date();  
alert(d.toDateString()); // Afficher une chaîne de date abrégée  
alert(d.toLocaleDateString()); // Afficher une chaîne de date localisée  
alert(d.toISOString()); // Afficher la chaîne de dates normalisées ISO  
alert(d.toUTCString()); // Afficher une chaîne de date convertie en heure UTC  
alert(d.toString()); // Afficher la chaîne de date complète avec le fuseau horaire local
```

De même, vous pouvez utiliser les méthodes `toLocaleTimeString()`, `toTimeString()` de l'objet Date pour générer des chaînes d'heure, comme illustré dans l'exemple suivant :

```
const d = new Date(); alert(d.toTimeString()); // Afficher la partie horaire de la date  
alert(d.toLocaleTimeString()); // Afficher une chaîne de temps localisée
```

Obtenir des composants de date et d'heure spécifiques

Une fois que vous avez un objet de date approprié, un certain nombre de méthodes sont à votre disposition pour en extraire des détails, tels que le mois, la date, la valeur des heures ou des minutes, etc. La section suivante décrit les différentes méthodes d'extraction d'informations individuelles à partir d'un objet. Objet Date.

Obtenir l'année, le mois et la date

L'objet Date fournit plusieurs méthodes telles que `getFullYear()`, `getMonth()`, `getDay()`, etc. que vous pouvez utiliser pour extraire les composants de date spécifiques de l'objet Date, tels que l'année, le jour du mois, le jour de la semaine, etc. respectivement. L'exemple suivant montre comment obtenir des composants de date spécifiques à partir de l'objet Date à l'aide de ces méthodes :

```
const d = new Date();  
// Extraction de la partie date  
alert(d.getDate()); // Afficher le jour du mois  
alert(d.getDay()); // Afficher le nombre de jours de la semaine (0-6)  
alert(d.getMonth()); // Afficher le nombre de mois de l'année (0-11)  
alert(d.getFullYear()); // Afficher l'année complète (quatre chiffres)
```

La méthode `getDay()` renvoie un nombre représentant le jour de la semaine (de 0 à 6) au lieu de renvoyer un nom tel que dimanche ou lundi de sorte que, si c'est dimanche, la méthode renvoie 0 ; et si c'est lundi , la méthode renvoie 1, et ainsi de suite.

De même, la méthode `getMonth()` renvoie le nombre de mois (de 0 à 11) au lieu du nom du mois. Ici 0 représente le premier mois de l'année. Par conséquent, s'il s'agit de janvier, la méthode renvoie 0 et non 1 ; et si c'est août, la méthode renvoie 7.

Consulter la documentation pour plus de méthode :

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Date